Document Processing Module

This document describes the ingestion and processing pipeline implemented in rag/document_loader.py, rag/text_cleaner.py, rag/chunking.py and rag/ingestion.py. Figure 1 shows the pipeline exactly as implemented.

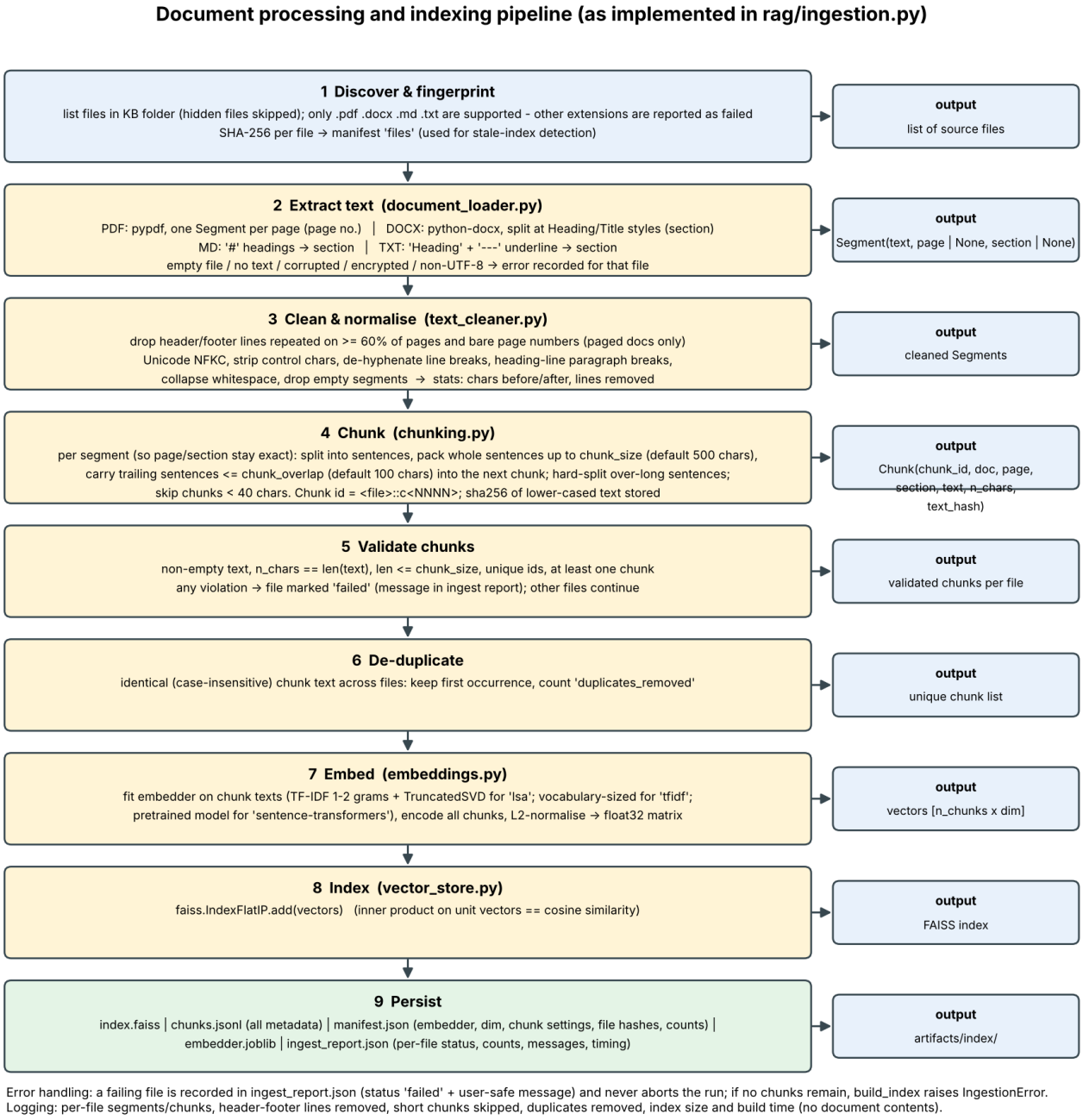


Figure 1. Document processing and indexing pipeline (generated from scripts/make_diagrams.py; Mermaid text version in diagrams/).

1. Supported formats and limitations

Format	Extraction	Metadata	Limitations
.pdf	pypdf, one segment per page	page number	No OCR: scanned PDFs have no text and are rejected; encrypted PDFs rejected; tables/columns may interleave; headings are detected heuristically.
.docx	python-docx paragraphs, split at Heading/Title styles	section	No page numbers; tables, text boxes and images ignored.
.md	'#' headings start sections	section	Markup inside text is kept as-is.
.txt	UTF-8; 'Heading' followed by '---' underline starts a section	section	Non-UTF-8 files rejected; headings otherwise undetected.

2. Cleaning and normalisation

- Repeated header/footer lines (normalised with digits masked) that occur on at least 60% of the pages of a paged document are removed, as are bare page numbers ('Page 3', '3 / 10'). Documents with fewer than two pages are not de-headed.
- Unicode NFKC normalisation, control-character removal, de-hyphenation of words broken across lines, single newlines joined, runs of whitespace collapsed.
- Heading heuristic: a short (<= 60 characters, <= 8 words), capitalised line without terminal punctuation, followed by a capitalised line, keeps its own paragraph so that headings are not glued to sentences.
- Empty segments are dropped; if no text remains the file is rejected (EmptyDocumentError).

3. Chunking strategy, size and overlap

Sentence-aware packing: text is split into sentences (regex on terminal punctuation followed by a capital letter or digit); whole sentences are added until the next sentence would exceed chunk_size (default 500 characters). The next chunk starts with the trailing sentences of the previous chunk that fit in chunk_overlap (default 100 characters), provided the overlap plus the next sentence still fits. Sentences longer than chunk_size are split at word boundaries. Chunking is applied per page/section so every chunk has exact page/section metadata (a paragraph that crosses a PDF page break is therefore split at the break). Rationale: intact sentences make extractive answers and citations readable; per-segment chunking preserves provenance. Trade-off: chunk lengths are uneven. Both parameters are configurable (RAG_CHUNK_SIZE, RAG_CHUNK_OVERLAP, or the CLI flags) and validated (size >= 100, 0 <= overlap < size).

4. Metadata preservation

Each Chunk carries chunk_id (<file>::cNNNN), doc (filename), page (PDFs), section, text, n_chars and a SHA-256 prefix of the lower-cased text (used for de-duplication). All fields are persisted in chunks.jsonl and shown in the UI source table.

5. Validation and error handling

Condition	Detected by	Behaviour
Unsupported extension	load_document	UnsupportedFormatError; file reported 'failed'; run continues
Empty file / whitespace only / scanned PDF	load_document	EmptyDocumentError; reported 'failed'
Corrupted PDF or DOCX, encrypted PDF, non-UTF-8 text	loader try/except	IngestionError with user-safe message (no stack trace shown); logged
Chunk problems (empty, oversize, duplicate id, none produced)	validate_chunks	File reported 'failed' with the first problem
Unexpected exception	process_corpus	Recorded as 'unexpected error (<type>)'; stack trace only in the log
Identical chunk text across files	text hash	First kept; count reported as duplicates_removed
No usable chunk at all / missing folder	build_index	IngestionError raised to the caller / UI

6. Logging and statistics

ingestion logs per file the segment and chunk counts, and records characters before/after cleaning, header/footer lines removed, empty segments dropped, short chunks skipped, duplicates removed and index build time. Logs contain file names and counts, never document text or secrets. The same statistics are saved in artifacts/index/ingest_report.json and shown in the UI.

7. Measured results on the sample corpus

File	Segments	Chunks	Chars before	Chars after	Hdr/ftr removed	Short skipped
01_rag_overview.pdf	2	7	2822	2728	4	0
02_text_chunking.md	5	5	1640	1631	0	0
03_embeddings_and_similarity.docx	5	5	1683	1683	0	0
04_vector_search_faiss.pdf	2	6	2160	2066	4	0
05_retrieval_metrics.md	5	5	1478	1469	0	0
06_prompting_and_grounding.docx	5	5	1609	1609	0	0
07_security_prompt_injection.md	4	4	1212	1205	0	0
08_lexical_vs_dense_retrieval.txt	5	4	1205	1201	0	1
09_index_maintenance_note.txt	4	4	609	606	0	0

Total: 45 chunks from 9 files; failures: 0. Failure handling is exercised by tests/test_ingestion_index.py (corrupted PDF, empty, unsupported, no usable documents, missing folder, duplicates).